Real Estate Management & Property Listing Platform

Production-Grade Project Description & Technology Stack

Frontend: React.js + JavaScript **Backend:** Node.js + Express.js **Database:** PostgreSQL / MySQL

1. Project Overview

The Real Estate Management & Property Listing Platform is a production-oriented web application for managing real estate properties, agents, customers, inquiries, appointments, documents, and listing workflows. It provides a public property-search experience together with secure administrative and agent portals. The platform is designed to support real business operations such as listing approval, lead assignment, customer follow-up, property viewing, and listing status management.

The application can begin as a single real-estate company platform and later be extended into a multi-branch or multi-tenant SaaS product.

2. Main Objectives

- Centralize property listings, agents, customers, and inquiries.
- Provide fast property search, filtering, sorting, and pagination.
- Give agents a secure workspace for managing listings and leads.
- Provide administrators with approval, user-management, reporting, and audit capabilities.
- Track the complete customer journey from inquiry to viewing and sale/rental.
- Provide a responsive public website suitable for desktop and mobile users.

3. User Roles

Role	Responsibilities
Super Admin	Full system configuration, users, roles, permissions, branches/organizations, and system settings.
Administrator	Manage properties, agents, customers, inquiries, approvals, reports, and operational settings.
Agent	Create and manage assigned listings, respond to inquiries, schedule viewings, and manage leads.
Property Manager	Monitor assigned properties, listing status, documents, appointments, and operational activity.
Customer / Buyer / Tenant	Search properties, view details, save favorites, submit inquiries, and request appointments.

4. Functional Modules

4.1 Property Management

Create, update, publish, archive, and manage properties. Each property can contain title, description, property type, sale/rental mode, price, address, location, bedrooms, bathrooms, floor area, lot area, amenities, status, images, floor plans, and supporting documents.

4.2 Property Search & Filtering

Provide keyword search, location filtering, property type, sale/rent mode, minimum and maximum price, bedrooms, bathrooms, floor area, lot area, amenities, listing status, sorting, and pagination. Search should be optimized through database indexes and efficient API queries.

4.3 Property Details

A public property page displays the image gallery, property description, specifications, amenities, location/map, pricing, listing status, agent information, inquiry form, appointment request, and similar properties.

4.4 Agent Management

Manage agent accounts, profiles, contact details, assigned properties, lead assignments, appointments, listing performance, and account status. Agent permissions are enforced by the backend.

4.5 Inquiry & Lead Management

Capture inquiries from property pages and other channels. Store customer information, interested property, message, lead source, assigned agent, status, notes, follow-up date, and interaction history.

Suggested lead lifecycle: New → Contacted → Qualified → Viewing Scheduled → Negotiation → Won / Lost.

4.6 Appointment & Property Viewing

Customers can request property viewings. Agents can manage availability, appointment dates, confirmation, rescheduling, cancellation, and completion.

4.7 Favorites & Saved Searches

Authenticated customers can save properties, create saved searches, and receive notifications when matching listings are added or updated.

4.8 Property Documents

Store and manage documents associated with properties, such as brochures, floor plans, permits, and other authorized attachments. Access should be controlled and files should not be exposed through predictable public URLs.

4.9 Listing Approval

Agents can submit listings for review. Administrators can approve, reject, request changes, publish, unpublish, or archive listings.

4.10 Dashboard & Analytics

Administrative dashboards show active listings, sold/rented properties, agents, new inquiries, pending approvals, appointments, lead conversion, and property performance.

5. Core Business Workflow

Agent creates property → listing submitted → administrator reviews → listing approved → property published → customer searches → customer views details → inquiry submitted → lead created → lead assigned to agent → follow-up → property viewing → negotiation → sold/rented.

6. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	React.js	Component-based web interface, dashboards, public property pages, forms, search, and porta
Frontend Language	JavaScript (ES6+)	The only frontend programming language. No TypeScript.
Frontend Routing	React Router	Public routes, protected routes, agent/admin portals, and property detail pages.
API Client	Axios or Fetch API	Communicate with the Express.js REST API.
State Management	React Context API or Redux Toolkit	Authentication state, user state, filters, favorites, and shared application data.
UI / Styling	CSS3 + Tailwind CSS	Responsive layout and reusable interface styling.
Backend Runtime	Node.js	Server-side JavaScript runtime.
Backend Framework	Express.js	REST API, routing, middleware, authentication, validation, and business logic.
Backend Language	JavaScript	Backend application logic and API implementation.
Database	PostgreSQL or MySQL	Relational storage for properties, users, agents, customers, leads, appointments, and audit log
ORM	Prisma or Sequelize	Database models, migrations, relations, transactions, and queries.
Authentication	JWT + secure password hashing	Login and protected API access.
Authorization	RBAC	Role and permission enforcement at the API layer.
Validation	Joi / Zod / express-validator	Validate API request data and business inputs.
File Storage	S3-compatible object storage	Property images, floor plans, brochures, and documents.
Maps	Google Maps API or Mapbox	Property locations, maps, markers, and location search.
PDF	Puppeteer or PDFKit	Property brochures, reports, and printable documents.
API Documentation	Swagger / OpenAPI	Document and test REST API endpoints.
Version Control	Git + GitHub/GitLab	Source control, collaboration, code review, and CI/CD.
Deployment	Docker + VPS / Cloud	Consistent production deployment and service management.
Monitoring	Logs + health checks	Error tracking, operational monitoring, and diagnostics.

7. Database Design

Core tables: users, roles, permissions, user_roles, agents, customers, properties, property_types, property_images, property_amenities, amenities, property_documents, property_locations, listings, listing_status_history, inquiries, inquiry_notes, lead_assignments, appointments, favorites, saved_searches, notifications, branches/organizations, and audit_logs.

Use foreign keys, indexes, unique constraints, timestamps, transactions, and appropriate archive/status fields. PostgreSQL is recommended for a new production deployment; MySQL is also a strong alternative and can use the same application architecture.

8. Important Data Relationships

Organization/Branch → Agents → Properties → Listings. Customers create inquiries against listings, inquiries can be assigned to agents, and appointments can be created from qualified leads. Property images and documents belong to properties, while listing status history records publishing and availability changes.

9. API Structure

/api/auth — login, registration, logout/session, password management

/api/users — users, roles, permissions

/api/agents — agent profiles and assignments

/api/properties — property CRUD, details, media, documents

/api/listings — publish, approve, reject, archive, status history

/api/search — property search, filters, sorting, pagination

/api/inquiries — customer inquiries and lead management

/api/appointments — viewing requests and schedules

/api/favorites — saved properties

/api/saved-searches — saved filters and matching listings

/api/notifications — notifications and announcements

/api/reports — property and business reports

/api/audit-logs — authorized activity history

10. Security Requirements

- Hash passwords using Argon2 or bcrypt; never store plain-text passwords.
- Enforce authentication and authorization on the backend for every protected operation.
- Validate all request bodies, query parameters, route parameters, and uploaded files.
- Use parameterized queries or an ORM to reduce SQL injection risk.
- Apply rate limiting to authentication and public inquiry endpoints.
- Restrict access to private property documents and administrative data.
- Use HTTPS/TLS in production and protect secrets with environment variables or a secret manager.
- Maintain audit logs for listing approvals, account changes, lead assignments, and important property changes.
- Implement database backups and regularly test restoration.

11. Development Roadmap

Phase 1 — Foundation: React project, Express API, database, authentication, RBAC, environment configuration.

Phase 2 — Property Management: property CRUD, property types, amenities, images, locations, statuses.

Phase 3 — Public Marketplace: property listing pages, search, filters, sorting, pagination, property details.

Phase 4 — Agent Portal: agent accounts, dashboards, listing management, profile, assignments.

Phase 5 — Lead CRM: inquiries, lead statuses, assignment, notes, follow-ups, customer history.

Phase 6 — Appointments: viewing requests, agent schedules, confirmations, cancellations.

Phase 7 — Documents & Reports: secure files, brochures, PDF reports, analytics.

Phase 8 — Production Hardening: testing, security review, indexes, caching where needed, monitoring, backups, Docker, deployment, and documentation.

12. Portfolio Positioning

Present this project as a **Real Estate Management & Property Listing Platform**, not simply a property CRUD application. Demonstrate the complete business workflow from property creation and approval through public discovery, customer inquiry, agent assignment, viewing, and final property status.

The project demonstrates professional full-stack JavaScript skills: React frontend development, Node.js/Express REST APIs, PostgreSQL/MySQL relational modeling, authentication and RBAC, advanced search and filtering, image/file management, maps integration, CRM-style lead management, appointment workflows, dashboards, PDF generation, auditability, and production deployment.

Final Stack: React.js + JavaScript (ES6+) → Node.js + Express.js → PostgreSQL or MySQL. TypeScript is intentionally not used.